

3주차

개요

1. GPU 서버 시스템 아키텍처 (Dual-CPU)

구성요소

2. NUMA (Non-Uniform Memory Access)

주요 개념

4. NUMA Locality

3. GPUDirect

GPUDirect P2P

GPUDirect Storage

GPUDirect RDMA

참고 자료

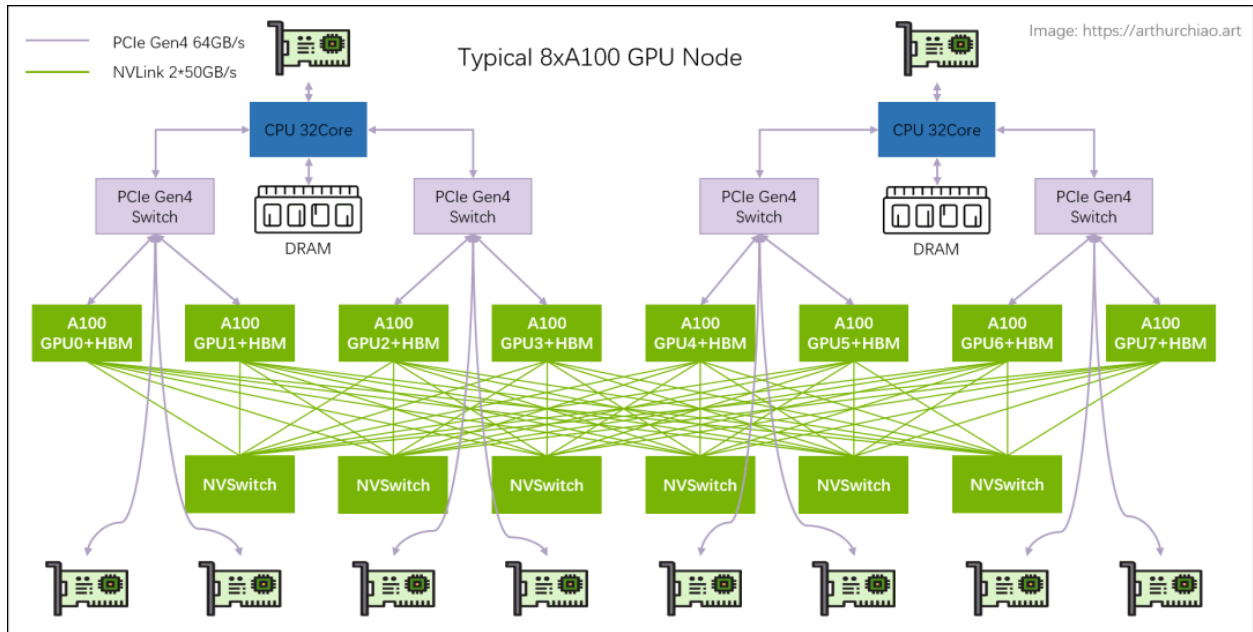
개요

이전 주차에서는 GPU의 내부 구조에 대해 학습했다. 하지만 이러한 GPU는 단독으로 동작할 수 없으며, 실제 병목 또한 GPU 외부에서 자주 발생한다.

따라서 서버 내 CPU, 메모리 PCI 버스 등과 어떤 방식으로 연결되어 있는지, 이러한 구조에서 중요하게 봐야 할 병목 발생 지점, 통신 방식 등을 이해해보고자 한다.

1. GPU 서버 시스템 아키텍처 (Dual-CPU)

Dual-CPU GPU 서버의 구조는 아래와 같다.



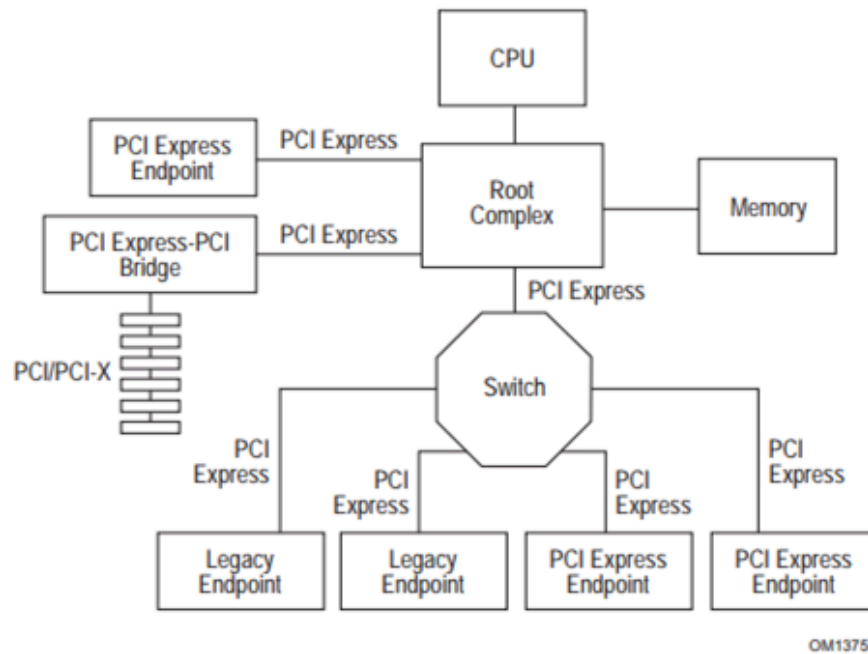
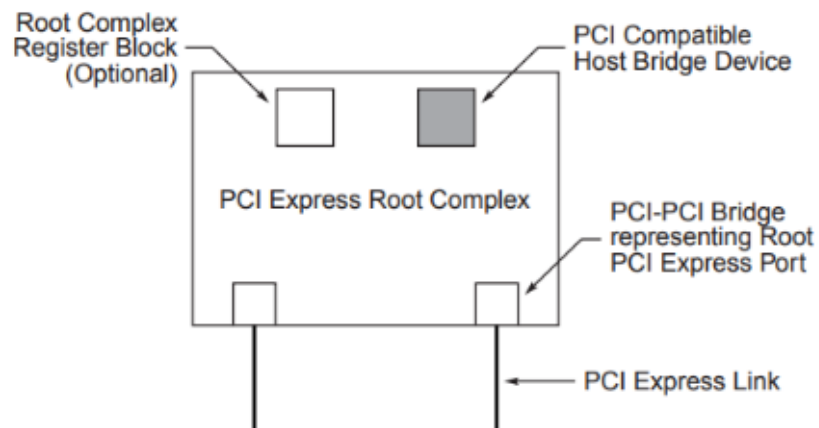


Figure 1-2: Example Topology

- **PCIe Root Port**



- PCIe Root Complex 내 논리적 포트이며, PCIe 버스가 시작되는 논리적 경계점이다.
- 시스템 부팅 시 BIOS가 각 Root Port 단위로 PCI Bus 번호를 부여하며, 각 Port는 독립적이다.

- **Memory Controller**

- CPU 칩 내부에서 **CPU Core** ↔ **DRAM** 사이를 연결하는 인터페이스이다.
 - 독립된 전용 버스로 DRAM과 연결된다.
 - **Host Memory**
 - CPU가 사용하는 일반적인 System Memory (DRAM)
-

2. NUMA (Non-Uniform Memory Access)

메모리에 접근하는 시간이 균일하지 않은 아키텍처를 의미한다. 단일 CPU 시스템에서는 모든 메모리가 하나의 CPU에 연결되어 있어 어느 메모리에 접근하든 거리가 같다.

하지만 멀티 프로세서 시스템의 경우, 각 CPU마다 직접적으로 연결된 메모리가 존재하고, CPU가 자신의 메모리에 접근하는 시간과 다른 CPU와 연결된 메모리에 접근하는 시간이 다르다.

이처럼 CPU가 메모리에 접근하는 경로·시간이 **메모리의 물리적 위치에 따라 달라지는** 아키텍처를 NUMA라고 한다.

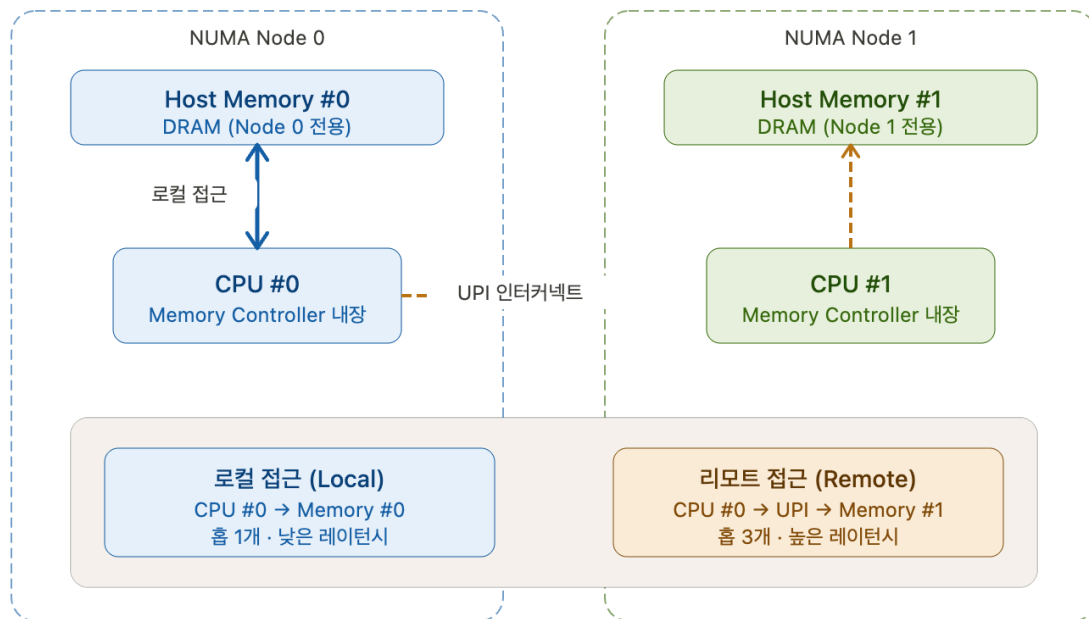
 **NUMA Node란?** 하나의 CPU 소켓과 해당 소켓에 직접 연결된 모든 구성요소들의 묶음을 의미한다. → CPU 코어, 캐시, 메모리 컨트롤러, 호스트 메모리, 루트 컴플렉스, PCIe 디바이스

따라서 NUMA는 여러 개의 NUMA Node로 구성되며, 각각 다른 노드의 구성 요소와 통신하는

주요 개념

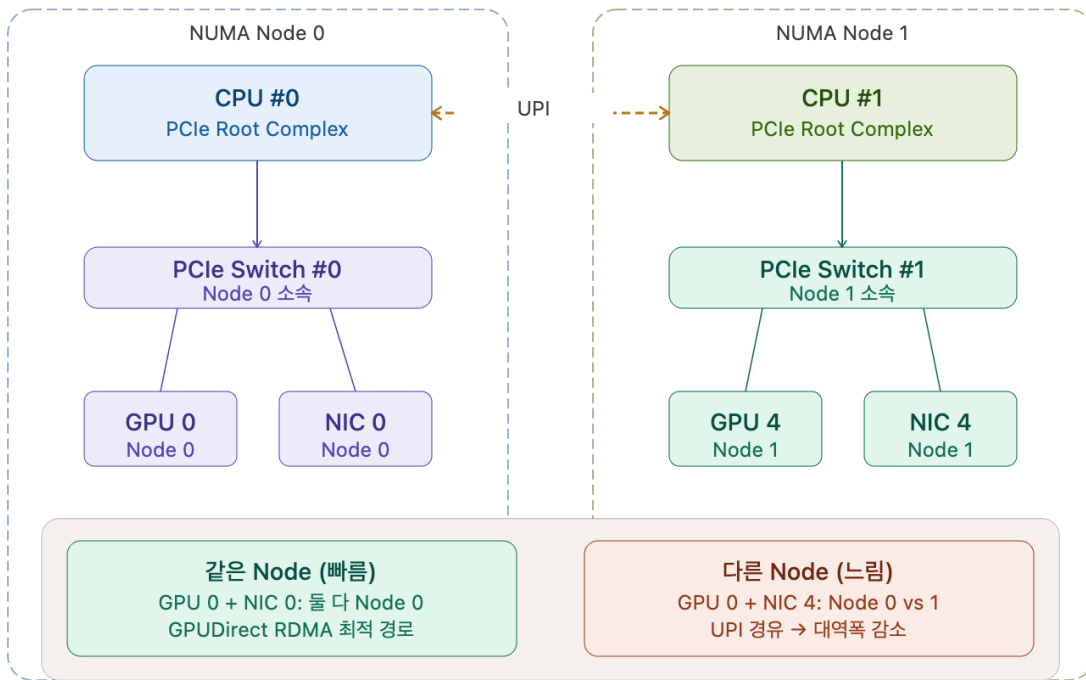
NUMA 아키텍처에서는 서버의 구성요소 간의 통신 과정에서 발생하는 **홉 수**에 따라 성능이 결정된다.

1. 지역성 (Locality)



- 각 CPU는 자신의 Memory Controller에 연결된 Host Memory에 접근하는 것이 더 빠르다.
 - CPU#0 → Memory Controller → Host Memory#0
- CPU#0가 다른 CPU#1과 연결된 Host Memory#1에 접근은 가능하나, **UPI(Ultra Path Interconnect)**를 거쳐야 한다.
 - 이 경우 대역폭이 줄고, 레이턴시가 늘어난다.

2. GPU, NIC와 같은 PCIe 디바이스도 NUMA의 영향을 받는다



- PCIe 디바이스가 어느 CPU의 PCIe Root Complex에 연결됐는지에 따라 NUMA Node가 결정된다.
- 다른 NUMA Node에 접근할 때 UPI를 거쳐야 하기에 성능 저하가 발생한다.
→ **UPI(Ultra Path Interconnect)**란, 두 CPU 소켓을 직접 연결하는 전용 고속 링크이다.

4. NUMA Locality

결국 NUMA 아키텍처에서는 홑 수에 따라 성능이 결정되는데, `nvidia-smi topo -m` 명령어를 활용하여 이 명령어가 출력하는 Label을 통해 홑 수와 경로를 기준으로 GPU와 NIC 간의 연결 수준을 알 수 있다.

Label	의미	예시
PIX	한 개의 PCIe bridge를 지나는 경로 (동일 PCIe Switch 하위)	GPU0 ~ NIC0
PXB	두 개 이상의 PCIe Bridge를 지나는 경로 (동일 PCIe Switch 하위, Host Bridge 경유)	GPU0 ~ NIC1
NODE	동일 NUMA 노드 내부, Host Bridge(Root Complex)를 지나는 경로 (동일 NUMA 노드, 다른 PCIe Switch)	GPU1 ~ NIC3

Label	의미	예시
SYS	다른 NUMA Node를 지나는 경로, UPI 인터커넥트 경유 (다른 NUMA 노드)	GPU2 ~ NIC4
NV#	NVLink로 직접 연결(PCIe 우회)	GPU0 ~ GPU1

3. GPUDirect

GPU와 NIC 사이의 **경로가 짧을수록 GPUDirect RDMA 성능이 높아지고**, 길어질수록 대역폭이 줄고 레이턴시가 늘어난다.

GPU Direct란, NIC 및 스토리지에서 GPU 메모리로 직접적인 읽기/쓰기 작업이 가능한 기술이다.

기존의 GPU 서버 내부에서는 아래와 같이 모든 데이터가 CPU와 Host Memory를 거쳐야 했다.

[기본 경로 - GPUDirect 없음]

스토리지/NIC/다른 GPU



→ 하지만 이 경우 CPU가 개입되고, Host Memory를 중간 버퍼로 계속 사용하게 되어 비효율적이다.

GPUDirect는 이러한 통신 과정을 단계적으로 줄여나갔다.

GPUDirect P2P

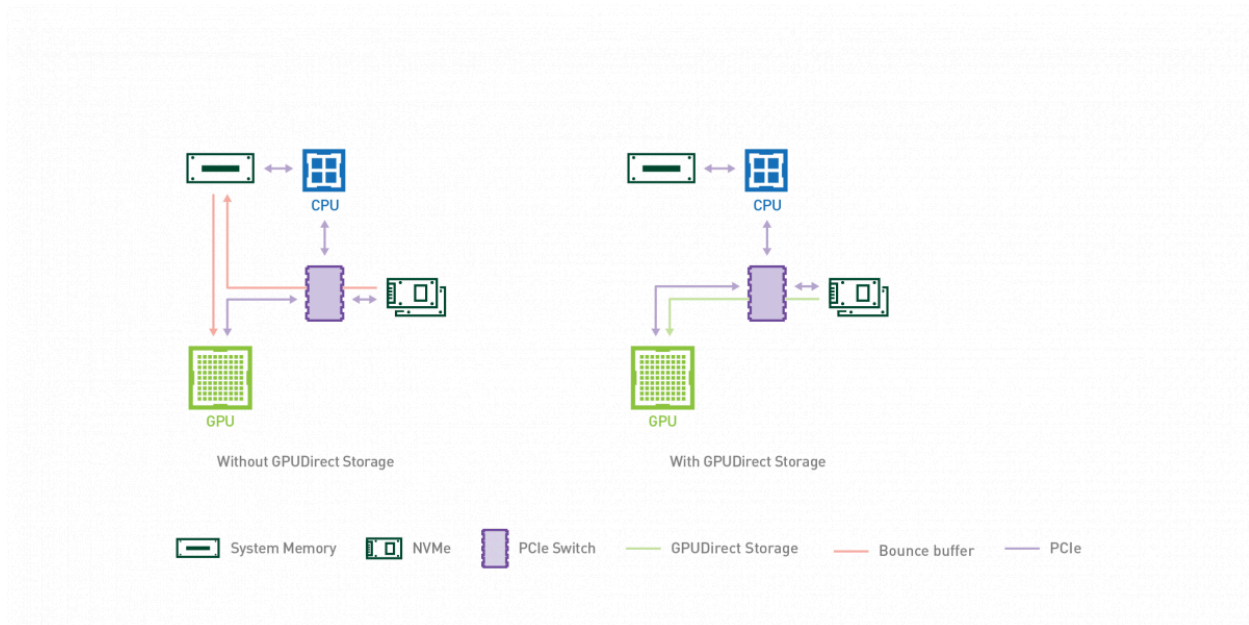
같은 서버 내 GPU끼리 Host Memory를 거치지 않고 직접 통신하는 것을 의미한다.

이는 두 GPU가 같은 PIX 관계를 갖거나 (같은 PCIe Switch에 존재) NVLink로 연결되어 있으면 활성화된다.

GPU #0 Memory → GPU #1 Memory
(PCIe 또는 NVLink로 직접 전송)

GPUDirect Storage

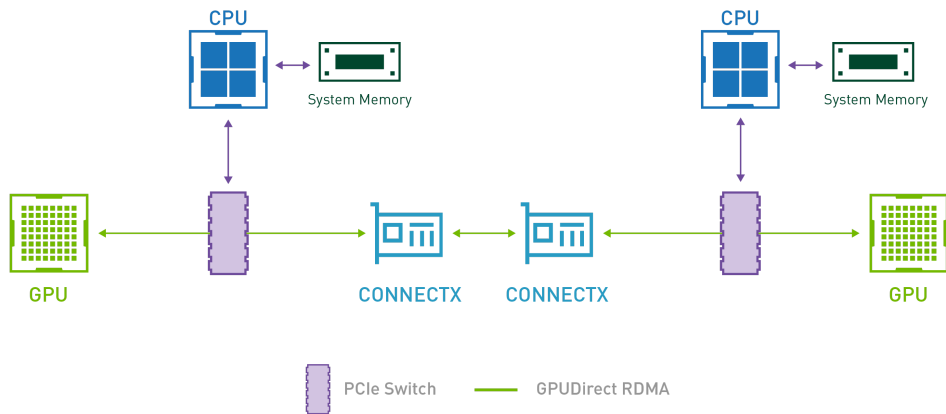
GPU가 Host Memory를 거치지 않고, 스토리지(NVMe SSD)에서 데이터를 직접 읽어오는 것을 의미한다.



AI 학습에서 데이터셋을 디스크에서 읽어 GPU로 올리는 작업이 반복되는데, 이 과정에서 CPU 개입이 줄고, Host Memory의 비효율적인 사용을 줄일 수 있다.

GPUDirect RDMA

Remote Direct Memory Access로, 네트워크를 통해 Host Memory, CPU를 완전히 우회하여 원격 GPU 메모리로 직접 전송하는 방식이다.



따라서 GPUDirect RDMA가 CPU와 Host Memory를 우회한 뒤의 PCIe 경로도 최대한 짧아야 하고, GPU와 NIC가 같은 PCIe Switch 하위에 있으면 **데이터가 Switch 하나만 통과하고 바로 NIC에 도달**하기 때문에 GPUDirect RDMA의 성능이 가장 높아진다.

참고 자료

- [Intel Xeon IIO](#)
- [PCIe Root Complex — PCI Express Base Specification](#)
- [NVIDIA NVLink and NVSwitch Supercharge LLM Inference](#)
- [3세대 NVIDIA NVSwitch를 통한 멀티-GPU 인터커넥트 업그레이드](#)
- https://www.fibermall.com/blog/gpu-server-topology-and-networking.htm?srsItd=AfmBOoozsVMnpdGSo3ISuH4_2VjdeT69w0BFdTqDw6bjMmFcp-juaoOt
- <https://towardsdatascience.com/how-gpus-communicate/>
- <https://chaimrand.medium.com/the-crucial-role-of-numa-awareness-in-high-performance-deep-learning-99ae3e8eb49a>

- <https://chaimrand.medium.com/the-crucial-role-of-numa-awareness-in-high-performance-deep-learning-99ae3e8eb49a>
- <https://developer.nvidia.com/gpudirect>